



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

Sprint 3

Grupo:

Jorge Carnicero Príncipe,

Andrés Gil Vicente,

Jorge González Pérez

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Índice

1. Introducción.....	3
1.1. Contexto.....	3
1.2. Objetivos del Sprint 3.....	3
1.3. Justificación.....	4
2. Metodología.....	5
2.1. Respuesta a HU-4: Arquitectura de Capas.....	5
2.2. Diseño de las Capas Bronze, Silver y Gold.....	5
2.2.1. Capa Bronze: Datos en Bruto.....	5
2.2.2. Capa Silver: Datos Procesados.....	6
2.2.3. Capa Gold: Indicadores Técnicos.....	7
2.3. Implementación Técnica con AWS Glue.....	8
2.3.1. Jobs de Transformación.....	8
2.3.2. Cálculo de Indicadores Técnicos.....	9
2.3.3. Orquestación del Pipeline.....	10
2.4. Configuración centralizada.....	11
3. Resultados.....	13
3.1. Descripción de los Resultados.....	13
3.2. Pantallazos de la Ejecución.....	14
3.3. Discusión de los Resultados.....	19
4. Conclusión.....	21
4.1. Resumen del Proceso.....	21
4.2. Principales Logros.....	21

1. Introducción

1.1. Contexto

El Sprint 3 representa la finalización de la fase de procesamiento batch del proyecto TradeData, construyendo sobre la infraestructura establecida en los sprints anteriores. En el Sprint 1 se implementó la capa de ingesta y almacenamiento de datos históricos en S3, y en el Sprint 2 se estableció el catálogo de metadatos mediante AWS Glue Data Catalog. Con estos cimientos en su lugar, el Sprint 3 se enfoca en la transformación y enriquecimiento de los datos mediante un pipeline completo de procesamiento distribuido.

Este sprint introduce la arquitectura Bronze-Silver-Gold. Esta arquitectura permite mantener la trazabilidad de las transformaciones y optimiza el rendimiento de consultas mediante formatos columnares como Parquet.

1.2. Objetivos del Sprint 3

Los objetivos principales de este sprint, derivados de la Historia de Usuario HU-4, son:

Implementación de Arquitectura Multi-Capa:

- Diseñar e implementar tres capas de datos diferenciadas: Bronze (datos en bruto), Silver (datos procesados) y Gold (datos analíticos)
- Organizar estas capas como carpetas separadas dentro del bucket principal de S3
- Establecer convenciones de nomenclatura y particionado coherentes entre capas

Transformación Bronze - Silver:

- Desarrollar un Job de AWS Glue que lea los archivos CSV de la capa Bronze
- Normalizar y limpiar los datos aplicando transformaciones básicas
- Convertir los datos al formato columnar Parquet para optimizar consultas analíticas
- Mantener el particionado por asset, año y mes en la capa Silver

Cálculo de Indicadores Técnicos (Silver - Gold):

- Implementar un Job de AWS Glue que calcule indicadores técnicos de trading sobre los datos de Silver
- Calcular los siguientes KPIs: Moving Average Simple (SMA 200), Moving Average Exponential (EMA 50), Relative Strength Index (RSI 14), Moving Average Convergence Divergence (MACD con señal e histograma)
- Almacenar los resultados en la capa Gold en formato Parquet particionado

Con posibilidad de añadir otros KPIs extras (lo cual hemos decidido hacer para seguir probando y explorando indicadores potencialmente útiles)

Automatización y Orquestación:

- Crear scripts de Python que automaticen la creación y configuración de los Jobs de Glue
- Implementar un pipeline orquestado que ejecute las transformaciones en el orden correcto
- Integrar la ejecución del Crawler para mantener el catálogo actualizado después de cada transformación
- Desarrollar un script maestro que permita ejecutar todo el proceso end-to-end con un solo comando

1.3. Justificación

La implementación de la arquitectura por capas y el procesamiento con AWS Glue es fundamental por múltiples razones. En primer lugar, separa claramente las responsabilidades de cada capa: Bronze almacena los datos originales inmutables permitiendo re-procesamiento, Silver proporciona datos limpios y normalizados para análisis general, y Gold contiene datasets especializados optimizados para casos de uso específicos como el análisis técnico de trading.

En segundo lugar, el uso de Apache Spark mediante AWS Glue permite procesar grandes volúmenes de datos de forma distribuida y escalable. Aunque actualmente solo procesamos SOLUSD, la arquitectura está preparada para escalar horizontalmente si se agregan todas las criptomonedas restantes sin modificaciones significativas en el código.

En tercer lugar, la conversión de CSV a Parquet no es simplemente un cambio de formato, sino una optimización crítica. Parquet es un formato columnar que permite leer solo las columnas necesarias en cada consulta, reduce significativamente el tamaño de almacenamiento mediante compresión eficiente, y mejora drásticamente el rendimiento de herramientas analíticas como Athena y QuickSight.

Finalmente, el cálculo de indicadores técnicos directamente en la capa de procesamiento evita cálculos repetitivos en herramientas de visualización, pre-computa métricas complejas que requieren ventanas temporales extensas (como SMA 200), y proporciona una base sólida para futuros modelos de machine learning que podrían utilizar estos indicadores como features.

2. Metodología

2.1. Respuesta a HU-4: Arquitectura de Capas

La HU-4 exige implementar una arquitectura de tres capas, Bronze–Silver–Gold, que organiza el Data Lake en etapas con responsabilidades claras. La capa **Bronze** funciona como zona de aterrizaje con datos en bruto, útil como punto de restauración ante errores. La capa **Silver** consolida una “single source of truth” con datos ya validados y normalizados. Por último, la capa **Gold** almacena agregaciones y cálculos especializados orientados al consumo (en este caso, indicadores).

En el proyecto, estas capas se han materializado como prefijos dentro del bucket `trade-data-big-daddyks-main`, con las rutas `bronze/`, `silver/prices/` y `gold/indicators/`, manteniendo un esquema consistente de particionado por asset, año y mes para facilitar consultas temporales.

La adaptación de `complete.py` respecto al Sprint 1 fue mínima pero efectiva: se añadió `LAYER_PREFIX = "bronze"` para anteponer automáticamente el prefijo a las claves S3, preservando la lógica previa de caché incremental y retención. Además, la creación inicial del bucket se ajustó para generar también los prefijos de las tres capas, creando las “carpetas” visibles mediante `put_object` con claves terminadas en `/` (por ejemplo, `bronze/`, `silver/` y `gold/`).

2.2. Diseño de las Capas Bronze, Silver y Gold

2.2.1. Capa Bronze: Datos en Bruto

La capa Bronze almacena los datos históricos exactamente como se obtienen de TradingView, sin transformaciones. Esta capa se implementó en el Sprint 1 y se ha actualizado en el Sprint 3 para incorporar el prefijo en la nomenclatura de claves S3. Las características principales de Bronze incluyen el formato CSV para mantener compatibilidad con múltiples herramientas, el uso de convenciones Hive partitioning con la estructura `asset={ASSET}/year={YYYY}/month={MM}/`, y el esquema original con las columnas `datetime`, `symbol`, `open`, `high`, `low`, `close` y `volume` sin validaciones ni transformaciones. La inmutabilidad es un principio fundamental: los archivos en Bronze nunca se modifican una vez creados, solo se agregan nuevas particiones mediante la lógica de carga incremental. Esta capa actúa como fuente de verdad primaria y permite re-procesamiento completo si es necesario sin pérdida de información.

El script mantiene toda la funcionalidad de caché implementada mediante la función `infer_last_ingested_from_s3`, que consulta el último archivo disponible en S3 para cada asset y calcula el delta temporal necesario. La gestión de retención se realiza mediante `retention_cleanup_months`, que elimina automáticamente particiones más antiguas que el período de 4 años configurado, aunque este comportamiento puede desactivarse mediante el booleano `ENABLE_RETENTION_CLEANUP`.

2.2.2. Capa Silver: Datos Procesados

La capa Silver contiene datos que han pasado por un proceso de limpieza, normalización y optimización de formato. Esta capa es el punto de partida recomendado para la mayoría de análisis y consultas.

Transformaciones aplicadas en Silver:

- **Conversión de formato:** De CSV row-based a Parquet columnar
- **Normalización temporal:** El campo datetime se convierte de string a tipo timestamp de Spark
- **Normalización de precios:** Los campos OHLC se convierten explícitamente a double para garantizar precisión numérica
- **Añadido de metadatos:** Se agrega columna `_ingest_ts` con timestamp de cuando se procesó el registro
- **Validación básica:** Se eliminan filas completamente vacías mediante `.dropna(how="all")`

Particionado en Silver:

El particionado se mantiene coherente con Bronze utilizando las columnas `asset`, `year` y `month`. Esta normalización facilita consultas case-insensitive y mejora la compatibilidad con diferentes herramientas que pueden interpretar las particiones de forma distinta. El particionado se especifica en Spark mediante `.partitionBy("asset", "year", "month")` al escribir el DataFrame.

Formato Parquet y compresión:

La elección de Parquet tiene muchas ventajas como:

- **Compresión eficiente:** Buen balance entre velocidad y ratio de compresión
- **Proyección de columnas:** Solo se leen las columnas necesarias en cada consulta (column pruning)
- **Predicado pushdown:** Los filtros se aplican a nivel de archivo antes de cargar datos en memoria
- **Compatibilidad universal:** Soportado nativamente por Spark, Athena, Presto, Hive, Pandas y la mayoría de herramientas Big Data

2.2.3. Capa Gold: Indicadores Técnicos

La capa Gold almacena datasets especializados optimizados para casos de uso específicos. En este sprint, se ha implementado una tabla Gold que contiene indicadores técnicos de trading calculados sobre los datos de Silver.

Indicadores implementados:

Los indicadores técnicos son herramientas matemáticas utilizadas en análisis técnico para predecir movimientos futuros de precios basándose en patrones históricos. Los cuatro indicadores implementados en este sprint son:

1. Simple Moving Average 200 (SMA 200):

- Promedio aritmético del precio de cierre de los últimos 200 días
- Se calcula como: $SMA(200) = (P1 + P2 + \dots + P200) / 200$
- Implementación en Spark: `close.rolling(window=200, min_periods=200).mean()`
- Interpretación: Indica la tendencia de largo plazo; precio por encima sugiere tendencia alcista

2. Exponential Moving Average 50 (EMA 50):

- Promedio ponderado exponencialmente que da más peso a precios recientes
- Fórmula recursiva: $EMA(t) = \alpha \times P(t) + (1-\alpha) \times EMA(t-1)$ donde $\alpha = 2/(N+1)$
- Implementación: `close.ewm(span=50, adjust=False, min_periods=50).mean()`
- Interpretación: Reacciona más rápido a cambios de precio que SMA; útil para tendencias de medio plazo

3. Relative Strength Index 14 (RSI 14):

- Oscilador de momento que mide la velocidad y magnitud de cambios de precio
- Rango: 0-100; valores >70 indican sobrecompra, <30 indican sobreventa
- Cálculo: $RSI = 100 - (100 / (1 + RS))$ donde $RS = \text{promedio de ganancias} / \text{promedio de pérdidas}$
- Implementación: Se calcula el delta de precios, se separan ganancias y pérdidas, se aplica media exponencial con $\alpha=1/14$

4. Moving Average Convergence Divergence (MACD):

- Indicador de tendencia que muestra la relación entre dos EMAs
- Componentes: MACD line = $EMA(12) - EMA(26)$, Signal line = $EMA(9)$ del MACD, Histogram = $MACD - \text{Signal}$
- Implementación: Calcula EMA12 y EMA26, resta para obtener MACD, aplica EMA9 sobre MACD para señal
- Interpretación: Cruce de MACD sobre señal sugiere compra; cruce bajo señal sugiere venta.

Estructura de datos en Gold:

La capa Gold conserva todas las columnas de Silver y añade las nuevas columnas calculadas: `sma_200`, `ema_50`, `rsi_14`, `macd`, `macd_signal`, `macd_hist`. El particionado se mantiene coherente (`asset`, `year`, `month`) para facilitar consultas temporales eficientes.

Indicadores extra implementados (ampliación GOLD)

Además de los indicadores requeridos, hemos decidido incorporar algunos KPIs adicionales orientados a análisis cuantitativo y control de riesgo, calculados sobre la serie temporal de `close` en la capa Gold:

- `log_return_1`: retorno logarítmico entre cierres consecutivos.
- `realized_vol_20_ann`: volatilidad realizada rolling 20, anualizada.
- `drawdown`: caída relativa del precio actual respecto al máximo acumulado histórico.
- `max_drawdown_252`: peor drawdown observado en una ventana rolling de 252 periodos (aproximadamente 1 año)
- `bb_percent_b_20_2`: posición normalizada del precio dentro de las Bandas de Bollinger

Actualización de la estructura en Gold:

La capa Gold conserva las columnas de Silver y añade, además de `sma_200`, `ema_50`, `rsi_14`, `macd`, `macd_signal`, `macd_hist`, las columnas extra: `log_return_1`, `realized_vol_20_ann`, `drawdown`, `max_drawdown_252`, `bb_percent_b_20_2`.

2.3. Implementación Técnica con AWS Glue

2.3.1. Jobs de Transformación

AWS Glue es un servicio serverless de ETL que permite ejecutar código Apache Spark sin gestionar infraestructura. Los Jobs de Glue proporcionan un entorno de ejecución escalable con workers configurables y permiten programar ejecuciones mediante triggers o ejecutarlos bajo demanda.

El Job Bronze a Silver, implementado en `job_bronze_to_silver.py`, ejecuta el primer paso de transformación del pipeline. El flujo comienza con la inicialización del Job mediante `getResolvedOptions` para resolver los argumentos pasados, la creación del contexto de Glue que extiende `SparkContext` con funcionalidades específicas, y la configuración de compresión Parquet mediante el parámetro `spark.sql.parquet.compression.codec` establecido en `snappy`. Se configura también el modo de sobrescritura dinámica de particiones con `spark.sql.sources.partitionOverwriteMode` para permitir actualizaciones granulares.

La lectura desde el Glue Catalog se realiza mediante `create_dynamic_frame_from_catalog`, especificando la base de datos y tabla origen. Este método acepta opcionalmente un predicado de filtrado a través del parámetro `push_down_predicate` para implementar partition pruning y reducir la cantidad de datos leídos. El `DynamicFrame` resultante se convierte a `DataFrame` de Spark para aplicar transformaciones estándar mediante el método `toDF`.

El proceso de normalización de datos implementa una lógica robusta de detección de columnas. Para la columna temporal, el script busca entre varios nombres candidatos (datetime, time, timestamp, date) y aplica la conversión a tipo timestamp mediante la función `to_timestamp`. De forma similar, normaliza la columna de precio de cierre buscando entre variantes (close, Close) y convierte explícitamente a tipo double. La columna asset se crea si no existe, extrayéndola de columnas existentes o utilizando el valor por defecto especificado en el parámetro `ASSET_DEFAULT`.

La validación de particiones es un paso crítico ya que el Crawler puede inferir las columnas year y month como tipo string desde los nombres de carpeta, mientras que necesitamos que sean integers para el particionado correcto. El script convierte explícitamente estas columnas al tipo apropiado. Finalmente, la escritura a Silver añade la columna de metadato `_ingest_ts` con el timestamp actual para trazabilidad, y escribe en formato Parquet con el modo especificado en el parámetro `WRITE_MODE`, típicamente append. El particionado se especifica mediante el método `partitionBy` con las columnas asset, year y month.

La parametrización del Job mediante DefaultArguments permite especificar la base de datos origen con `SRC_DB`, la tabla con `SRC_TABLE`, la ruta de destino con `SILVER_TARGET_PATH`, el modo de escritura con `WRITE_MODE`, las columnas de particionado con `PARTITION_COLS` separadas por comas, el asset por defecto con `ASSET_DEFAULT`, y opcionalmente un predicado de filtrado con `PUSH_DOWN`. Esta parametrización permite reutilizar el mismo Job para procesar diferentes assets o períodos temporales sin modificar el código.

2.3.2. Cálculo de Indicadores Técnicos

El Job Silver a Gold, implementado en `job_silver_to_gold_kpis.py`, calcula indicadores técnicos utilizando Pandas UDFs de Spark. Esta estrategia permite agrupar los datos por asset y aplicar funciones de Pandas sobre cada grupo de forma distribuida.

La lectura de datos Silver se realiza directamente desde la ruta S3 utilizando `spark.read.parquet`, sin pasar por el catálogo ya que la lectura directa de Parquet es más eficiente. Se validan la presencia de columnas requeridas (asset, datetime, close) y se normalizan los tipos de datos convirtiendo datetime a timestamp y close a double para garantizar precisión numérica en los cálculos posteriores.

El núcleo del Job utiliza Pandas UDFs decoradas con `@pandas_udf` especificando el esquema de salida y el tipo `PandasUDFType.GROUPED_MAP`. Esta función recibe un Pandas DataFrame con los datos de un asset específico, los ordena temporalmente para garantizar secuencia correcta, y aplica los cálculos de indicadores. La función `add_kpis` implementa los cuatro indicadores requeridos utilizando las capacidades vectorizadas de Pandas.

Para el SMA 200, se utiliza el método `rolling` con window de 200 períodos y `min_periods=200` para evitar valores espurios al inicio de la serie. El EMA 50 se calcula mediante `ewm` con span de 50 y el parámetro `adjust=False` que utiliza la fórmula recursiva estándar.

El RSI 14 requiere un cálculo más elaborado: primero se obtiene el delta de precios mediante `diff`, se separan ganancias y pérdidas utilizando operaciones vectorizadas de NumPy, se aplica media exponencial móvil con alpha igual a 1/14 sobre cada serie, se calcula el ratio de ganancias sobre pérdidas reemplazando ceros por NaN para evitar divisiones inválidas, y finalmente se aplica la fórmula estándar del RSI. El MACD se implementa calculando EMA de 12 y 26 períodos, restándolos para obtener la línea MACD, aplicando EMA de 9 períodos sobre el MACD para obtener la señal, y calculando el histograma como diferencia entre MACD y señal.

La aplicación distribuida se realiza mediante el método `groupBy` sobre la columna `asset` seguido de `apply` con la función de Pandas UDF. Spark distribuye automáticamente los grupos entre los workers disponibles y ejecuta la función en paralelo.

La escritura a Gold utiliza formato Parquet, particiona por las mismas columnas (`asset`, `year`, `month`) manteniendo coherencia con las capas anteriores, y utiliza el modo configurable especificado en `WRITE_MODE`. El esquema de salida incluye todas las columnas originales de Silver más las seis nuevas columnas calculadas: `sma_200`, `ema_50`, `rsi_14`, `macd`, `macd_signal` y `macd_hist`.

Extensión del Job Silver → Gold para KPIs extra

El mismo enfoque basado en cálculo vectorizado sobre series temporales se amplía incorporando KPIs adicionales de retorno y riesgo (retorno logarítmico, volatilidad realizada anualizada, drawdown y máximo drawdown rolling 252), así como un oscilador de bandas de volatilidad (Bollinger %B 20,2). Estos KPIs los calculamos sobre `close` y se incorporan como columnas adicionales en el dataset final almacenado en Gold.

2.3.3. Orquestación del Pipeline

La orquestación del pipeline completo se implementa mediante varios scripts que coordinan la ejecución secuencial de Jobs y Crawlers.

El Job `job_run_crawler.py` no procesa datos: su función es orquestar la ejecución del Crawler dentro del pipeline de Glue. Esto es necesario porque tras cada transformación que genera nuevos datos en S3, el Crawler debe ejecutarse para actualizar el catálogo. El flujo implementado verifica o crea la definición del Crawler mediante las funciones `get_crawler` y `create_crawler` o `update_crawler` según corresponda, especificando los targets S3 mediante el parámetro `Targets` que acepta una lista de paths. El Job espera a que el Crawler esté en estado READY mediante polling del campo `State`, lanza el Crawler con `start_crawler`, y opcionalmente espera a que complete si el parámetro `WAIT` está en true.

El script `pipeline_launcher.py` actúa como orquestador maestro del pipeline. Su primera responsabilidad es crear o actualizar todos los Jobs de Glue mediante la función `ensure_job`, que implementa una estrategia de borrado y recreación para garantizar que los DefaultArguments se actualicen correctamente.

La configuración de cada Job especifica el rol IAM mediante el parámetro `Role`, la ubicación del script con `ScriptLocation` apuntando a S3, los argumentos por defecto en `DefaultArguments` que incluyen todos los parámetros necesarios como rutas origen/destino y configuraciones más minuciosas.

La gestión del Crawler se realiza con `ensure_crawler`, configurando targets para las tres capas (bronze, silver y gold) mediante múltiples `S3Targets`, una `SchemaChangePolicy` en `LOG` (registrar cambios sin aplicarlos automáticamente) y una `RecrawlPolicy` en `CRAWL_NEW_FOLDERS_ONLY` para optimizar ejecuciones incrementales.

La ejecución del pipeline sigue una secuencia estricta: ejecutar el Crawler para catalogar Bronze, lanzar Bronze-to-Silver y esperar; ejecutar el Crawler para Silver; lanzar Silver-to-Gold y esperar; y ejecutar el Crawler una tercera vez para Gold. Esta secuencia garantiza que cada capa esté catalogada antes de ser consumida por el siguiente Job. La función `start_job_and_wait` implementa el patrón de espera, obteniendo el `JobRunId` al lanzar el Job, implementando polling del estado cada 20 segundos mediante `get_job_run`, verificando estados terminales (`SUCCEEDED`, `FAILED`, `STOPPED`, `TIMEOUT`), y lanzando excepciones si el Job termina en estado de fallo con el mensaje de error correspondiente.

Por último, `run_all.py` permite una ejecución end-to-end con un solo comando: valida/crea el bucket (`ensure_bucket_exists`), crea prefijos visuales y el directorio de scripts (`seed_prefixes`), sube los scripts al prefijo `scripts/` en S3 (`upload_scripts`), ejecuta `complete.py` (ingesta a Bronze) y después `pipeline_launcher.py` (proceso a través de Silver y Gold), ambos mediante `subprocess`. Esta automatización mejora la reproducibilidad y facilita testing y despliegues entre entornos.

2.4. Configuración centralizada

Con el objetivo de mantener el proyecto mantenible, consistente y fácil de desplegar, se ha optado por centralizar toda la configuración en el archivo `constants.py`.

En `constants.py` se agrupan las constantes en cuatro bloques principales:

1) Configuración de infraestructura

Se definen los parámetros básicos del entorno de ejecución, como la región de AWS y el bucket principal del Data Lake.

2) Prefijos de capas del Data Lake

Se establecen las rutas base en S3 para cada capa (Bronze/Silver/Gold), de modo que los jobs construyan sus lecturas y escrituras siempre desde un único origen de verdad.

3) Recursos de AWS Glue

Se parametrizan los recursos de Glue necesarios para el catálogo y la ejecución del pipeline: nombre de base de datos, crawler, prefijo de tablas y nombres de los jobs asociados.

4) Ubicaciones de scripts y roles IAM

Para facilitar la ejecución reproducible, se definen las rutas de los scripts en S3 y los roles IAM se obtienen desde variables de entorno (evitando llegar a poder a subir las credenciales al repo).

Ventajas:

- Permite cambiar nombres de recursos (bucket, jobs, crawlers, rutas) sin modificar el código de los jobs.
 - Facilita la portabilidad entre entornos, ajustando únicamente `constants.py`.
 - Evita hardcodear ARNs o credenciales en el código fuente, reduciendo riesgos de seguridad y mejorando las prácticas de despliegue.
-

3. Resultados

3.1. Descripción de los Resultados

La ejecución completa del pipeline del Sprint 3 generó exitosamente las tres capas de la arquitectura Bronce-Plata-Oro en Amazon S3. Los resultados principales que hemos obtenido incluyen:

Estructura de datos en S3:

- **Capa Bronze:** Mantiene los distintos archivos CSV mensuales de SOLUSD generados en el Sprint 1, ahora organizados bajo el prefijo **bronze/**
- **Capa Silver:** Se generaron archivos Parquet particionados conteniendo los datos normalizados de SOLUSD, siendo más rápido su acceso y lectura en comparación de los csv.
- **Capa Gold:** Se crearon archivos Parquet enriquecidos con nuevas columnas que incluyen indicadores técnicos, tanto los propuestos por la historia se usuario, como otros adicionales. Los KPIs pedidos son: SMA 200, EMA 50, RSI 14, MACD, MACD Signal, MACD Histogram y los KPIs extra son: log_return_1, realized_vol_20_ann, drawdown, max_drawdown_252, bb_percent_b_20_2

Catálogo de Glue actualizado:

El Crawler actualizó automáticamente el catálogo tras cada transformación, generando tres tablas, que se albergan dentro de la base de datos de **trade_data_imat3a05**:

- **lake_bronze**
- **lake_silver**
- **lake_gold**

Cada tabla incluye metadatos completos de esquema, tipos de datos, ubicación S3 y particiones reconocidas.

Jobs de Glue creados:

Se crearon y configuraron 3 Jobs en AWS Glue:

- **bronze_to_silver_imat3a05:** Job de transformación CSV - Parquet
- **silver_to_gold_imat3a05:** Job de cálculo de KPIs
- **run_crawler_imat3a05:** Job de orquestación del Crawler

3.2. Pantallazos de la Ejecución

Estructura de capas en S3:

General purpose buckets (1) Info

Buckets are containers for data stored in S3.

Find buckets by name

< 1 >

Name

AWS Region

Creation date

trade-data-big-daddyks-main

Europe (Spain) eu-south-2

February 11, 2026, 19:45:12 (UTC+01:00)

Amazon S3 > Buckets > trade-data-big-daddyks-main

Amazon S3

Buckets

Buckets de uso general

Buckets de directorio

Buckets de tablas

Buckets vectoriales

Seguridad y administración de acceso

Puntos de acceso

Puntos de acceso para FSx

Concesiones de acceso

Analizador de acceso de IAM

Información y administración de almacenamiento

Storage Lens

trade-data-big-daddyks-main Información

Objetos

Metadatos

Propiedades

Permisos

Objetos (4)

Los objetos son las entidades fundamentales que se almacenan en Ar
obtengan acceso a sus objetos, tendrá que concederles permisos de fi

Buscar objetos por prefijo

Nombre

Tipo

bronze/

Carpeta

gold/

Carpeta

scripts/

Carpeta

silver/

Carpeta

Contenido de la capa Bronze:

Amazon S3 > Buckets > trade-data-big-daddyks-main > bronze/ > asset=SOLUSD/ > year=2023/ > month=02/

Amazon S3

Buckets

Buckets de uso general

Buckets de directorio

Buckets de tablas

Buckets vectoriales

Seguridad y administración de acceso

Puntos de acceso

Puntos de acceso para FSx

Concesiones de acceso

Analizador de acceso de IAM

month=02/

Objetos

Propiedades

Objetos (1)

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el [inventario de Amazon S3](#) para
obtengan acceso a sus objetos, tendrá que concederles permisos de forma explícita. [Más información](#)

Buscar objetos por prefijo

Nombre

Tipo

Última modificación

data.csv

csv

12 Feb 2026 7:06:02 PM CET

14

Contenido de la capa Silver:

Amazon S3

▼ Buckets

Buckets de uso general

Buckets de directorio

Buckets de tablas

Buckets vectoriales

▼ Seguridad y administración de acceso

Puntos de acceso

Puntos de acceso para FSx

Concesiones de acceso

Analizador de acceso de IAM

▼ Información y administración de almacenamiento

Storage Lens

Operaciones por lotes

Configuración de la cuenta y la organización

► AWS Marketplace para S3

month=12/

ObjetosPropiedades

Objetos (3)

Copiar URI de S3

Copiar URL

Descargar

Abrir

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede utilizar el [inventario de Amazon S3](#) para obtener acceso a sus objetos, tendrá que concederles permisos de forma explícita. [Más información](#)

Buscar objetos por prefijo

☐	Nombre	Tipo	Última modificación
☐	part-00026-74e61d07-09f5-4a42-80ab-006d86496388.c000.snappy.parquet	parquet	18 Feb 2026 7:32:12 PM CET
☐	part-00026-7c0b33ab-1522-4864-8cda-f30f08c69984.c000.snappy.parquet	parquet	11 Feb 2026 7:49:12 PM CET
☐	part-00026-f35053aa-8aa9-4a53-94c1-1aa0a2a3948f.c000.snappy.parquet	parquet	18 Feb 2026 6:55:45 PM CET

Contenido de la capa Gold:

Amazon S3

▼ Buckets

Buckets de uso general

Buckets de directorio

Buckets de tablas

Buckets vectoriales

▼ Seguridad y administración de acceso

Puntos de acceso

Puntos de acceso para FSx

Concesiones de acceso

Analizador de acceso de IAM

▼ Información y administración de almacenamiento

Storage Lens

Operaciones por lotes

Configuración de la cuenta y la organización

► AWS Marketplace para S3

month=12/

ObjetosPropiedades

Objetos (3)

Copiar URI de S3

Copiar URL

Los objetos son las entidades fundamentales que se almacenan en Amazon S3. Puede obtener acceso a sus objetos, tendrá que concederles permisos de forma explícita

Buscar objetos por prefijo

☐	Nombre	Tipo
☐	part-00000-814b5909-8db6-4d06-ba1c-262c609c0c71.c000.snappy.parquet	parquet
☐	part-00000-92ffd39e-430e-4c94-bcfa-6688ea2f5772.c000.snappy.parquet	parquet
☐	part-00000-b91940c4-d55a-4a8b-803b-ad15af9b373c.c000.snappy.parquet	parquet

Jobs de Glue en la consola:

AWS Glue > Jobs

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Data Catalog

Data Integration and ETL

What's New ⓘ

Documentation ⓘ

AWS Marketplace

Enable compact mode

Enable new navigation

AWS Glue Studio Info

Create job Info

Author in a visual interface focused on data flow.

Visual ETL

Author using an interactive code notebook.

Notebook

Author code with a script editor.

Script editor

Example jobs Info

Create example job

Your jobs (3) Info

Filter jobs by property

☐	Job name	Type	Created by	Last modified	AWS Glue version
☐	run_crawler_imat3a05	Glue ETL	Script	18/2/2026, 19:22:41	4.0
☐	silver_to_gold_imat3a05	Glue ETL	Script	18/2/2026, 19:22:41	4.0
☐	bronze_to_silver_imat3a05	Glue ETL	Script	18/2/2026, 19:22:41	4.0

Crawlers creados:

AWS Glue > Crawlers

Getting started

ETL jobs

Visual ETL

Notebooks

Job run monitoring

Data Catalog tables

Data connections

Workflows (orchestration)

Data Catalog

Data Integration and ETL

What's New ⓘ

Documentation ⓘ

AWS Marketplace

Enable compact mode

Enable new navigation

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (3) Info

Last updated (UTC)
February 19, 2026 at 18:02:01

Action

Run

Create crawler

View and manage all available crawlers.

Filter crawlers

☐	Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from ...
☐	lake_bronze_imat3a05	Ready		Succeeded	February 18, 2026 at ...	View log	1 updated
☐	lake_gold_imat3a05	Ready		Succeeded	February 18, 2026 at ...	View log	1 updated
☐	lake_silver_imat3a05	Ready		Succeeded	February 18, 2026 at ...	View log	1 updated

Scripts ejecutados:

scripts/

Objects

Properties

Copy S3 URI

Objects (4)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

☐	Name	Type	Last modified	Size	Storage class
☐	job_bronze_to_silver.py	py	February 11, 2026, 19:45:13 (UTC+01:00)	3.2 KB	Standard
☐	job_run_crawler.py	py	February 11, 2026, 19:45:13 (UTC+01:00)	2.2 KB	Standard
☐	job_silver_to_gold_kpis.py	py	February 11, 2026, 19:45:13 (UTC+01:00)	3.2 KB	Standard
☐	pipeline_launcher.py	py	February 11, 2026, 19:45:13 (UTC+01:00)	5.2 KB	Standard

Base de datos de Glue:

Databases (1)

A database is a set of associated table definitions, organized into a logical group.

Filter databases

☐	Name	Description	Location URI	Source catalog	Created on (UTC)
☐	trade_data_imat3a05	-	-	256876712551	February 11, 2026 at 18:46:14

16

Tablas catalogadas en Glue:

trade_data_imat3a05

Database properties

Name	Description	Location
trade_data_imat3a05	-	-

Tables (3)

View and manage all available tables.

☐	Name	Database	Location	Classification
☐	lake_bronze	trade_data_imat3a05	s3://trade-data-big-daddyks-main/t	CSV
☐	lake_gold	trade_data_imat3a05	s3://trade-data-big-daddyks-main/ç	Parquet
☐	lake_silver	trade_data_imat3a05	s3://trade-data-big-daddyks-main/s	Parquet

Esquema de ficheros csv Bronze:

Schema (10)

View and manage the table schema.

Edit schema as JSON

Edit schema

#	Column name	Data type	Partition key	Comment
1	datetime	string	-	-
2	symbol	string	-	-
3	open	double	-	-
4	high	double	-	-
5	low	double	-	-
6	close	double	-	-
7	volume	double	-	-
8	asset	string	Partition (0)	-
9	year	string	Partition (1)	-
10	month	string	Partition (2)	-

Esquema de ficheros parquet Silver:

Schema (11)

View and manage the table schema.

Edit schema as JSON

Edit schema

#	Column name	Data type	Partition key	Comment
1	datetime	timestamp	-	-
2	symbol	string	-	-
3	open	double	-	-
4	high	double	-	-
5	low	double	-	-
6	close	double	-	-
7	volume	double	-	-
8	_ingest_ts	timestamp	-	-
9	asset	string	Partition (0)	-
10	year	string	Partition (1)	-
11	month	string	Partition (2)	-

Esquema de tabla Gold con KPIs:

Schema (22)

View and manage the table schema.

[Q](#)
[Filter schemas](#)

[Edit schema as JSON](#)
[Edit schema](#)

[<](#)
[1](#)
[2](#)
[>](#)
[⌘](#)

#	Column name	Data type	Partition key	Comment
1	datetime	timestamp	-	-
2	symbol	string	-	-
3	open	double	-	-
4	high	double	-	-
5	low	double	-	-
6	close	double	-	-
7	volume	double	-	-
8	_ingest_ts	timestamp	-	-
9	sma_200	double	-	-
10	ema_50	double	-	-
11	rsi_14	double	-	-
12	macd	double	-	-
13	macd_signal	double	-	-
14	macd_hist	double	-	-
15	log_return_1	double	-	-
16	realized_vol_20_ann	double	-	-
17	drawdown	double	-	-
18	max_drawdown_252	double	-	-
19	bb_percent_b_20_2	double	-	-
20	asset	string	Partition (0)	-

[illegible]

```
[15]: for col in parquetFile.columns:
      print(col)
```

```
datetime
symbol
open
high
low
close
volume
_ingest_ts
sma_200
ema_50
rsi_14
macd
macd_signal
macd_hist
log_return_1
realized_vol_20_ann
drawdown
max_drawdown_252
bb_percent_b_20_2
```

3.3. Discusión de los Resultados

Comportamiento del sistema:

El pipeline completo se ejecutó sin errores, procesando exitosamente los registros requeridos de la criptomoneda SOLUSD a través de las tres capas. El flujo de datos Bronze → Silver → Gold funcionó como se diseñó, con cada Job completándose en estado SUCCEEDED y el Crawler actualizando correctamente el catálogo después de cada transformación.

La estrategia de recrear Jobs (delete + create) en lugar de solo actualizarlos, garantiza que todos los DefaultArguments se reflejaran correctamente, evitando un bug común donde `update_job()` no siempre aplica cambios en argumentos. Esta decisión de diseño, aunque añade overhead al despliegue, mejora significativamente la fiabilidad del sistema.

Optimización de formato y tamaño:

Los archivos Parquet en Silver y Gold incluyen metadatos estadísticos (min/max/count por column por row group) que permiten técnicas avanzadas de optimización como predicate pushdown y column pruning, que serán aprovechadas automáticamente cuando se consulten mediante Athena en sprints futuros.

Cálculo de indicadores técnicos:

Los indicadores técnicos se calcularon correctamente sobre toda la serie temporal de SOLUSD. La estrategia de usar Pandas UDFs agrupadas por asset permitió aplicar rolling windows y exponential moving averages de forma eficiente, aprovechando las optimizadas implementaciones vectorizadas de NumPy y Pandas.

Un aspecto importante observado es que los primeros registros de indicadores como SMA 200 contienen valores NaN (no disponibles) porque requieren 200 observaciones previas para su cálculo. Esto es esperado y correcto: por ejemplo, para el dataset que comienza el 24 de enero de 2022, el primer valor válido de SMA 200 aparecerá aproximadamente en agosto de 2022. Herramientas de visualización deberán manejar estos NaN apropiadamente, típicamente omitiéndolos o interpolándolos según el caso de uso.

Particionado y performance:

El mantenimiento del esquema de particionado coherente (asset/year/month) a través de las tres capas facilita enormemente consultas temporales. Una consulta que solicite datos de SOLUSD de un mes específico solo leerá el archivo Parquet correspondiente a ese mes, ignorando los otros 48 meses. Este partition pruning reduce el tiempo de consulta en un factor de 50x en el mejor caso.

Sin embargo, se identificó una oportunidad de optimización futura: el particionado por mes puede ser subóptimo para análisis que requieran ventanas temporales específicas (ej. "últimos 90 días"). Para estos casos, podría evaluarse en sprints futuros un re-particionado por trimestre o la implementación de tablas agregadas adicionales en Gold optimizadas para rangos temporales comunes.

Escalabilidad observada:

Aunque actualmente solo procesamos SOLUSD, la arquitectura demostró estar preparada para escalar horizontalmente. Los Jobs utilizan la columna `asset` para particionar y agrupar datos, por lo que agregar las 12 criptomonedas restantes simplemente implicaría:

- Modificar `ASSETS = [ "SOLUSD" ]` a la lista completa en `complete.py`
- Re-ejecutar el pipeline end-to-end
- El procesamiento se distribuiría automáticamente entre workers

El tiempo de ejecución escalaría aproximadamente de forma lineal con el número de assets, pero el paralelismo de Spark y la distribución por particiones mantendría el tiempo dentro de rangos manejables.

Comparación con expectativas:

Los resultados se alinean estrechamente con las expectativas basadas en la HU-4. Los indicadores técnicos solicitados (SMA 200, EMA 50, RSI 14, MACD) se calcularon correctamente y están disponibles en la capa Gold, además de los KPIs adicionales que ya se han comentado. La arquitectura de capas se implementó siguiendo una estructura adecuada para mantener el orden y la jerarquía de carpetas más intuitiva y funcional.

Una observación positiva fue la robustez del sistema frente a re-ejecuciones. Al ejecutar el pipeline múltiples veces (durante desarrollo y testing), el modo `append` con particionado dinámico evitó duplicados: Spark sobrescribe solo las particiones afectadas, manteniendo intactas las particiones no tocadas.

4. Conclusión

4.1. Resumen del Proceso

El Sprint 3 culmina la implementación de la infraestructura de procesamiento batch del proyecto TradeData. El proceso comenzó con el diseño conceptual de la arquitectura de tres capas, seguido por la implementación de Jobs de AWS Glue utilizando Apache Spark para las transformaciones Bronze → Silver y Silver → Gold.

La fase de implementación incluyó el desarrollo de scripts de orquestación que automatizan completamente el despliegue y ejecución del pipeline. El script `pipeline_launcher.py` gestiona la creación de Jobs y su ejecución secuencial, mientras que `run_all.py` proporciona ejecución end-to-end con un solo comando, desde la ingesta de datos hasta el cálculo de indicadores técnicos.

La validación del correcto funcionamiento del sistema la hemos realizado mediante múltiples ejecuciones completas del pipeline, verificando tanto la correctitud de los datos transformados como la eficiencia del procesamiento. Los tests confirmaron que el sistema maneja correctamente escenarios de re-ejecución, actualización incremental y recuperación ante fallos.

4.2. Principales Logros

Los logros principales del Sprint 3 incluyen:

1. **Arquitectura de capas implementada:** Separación clara de datos en bruto (Bronze), procesados (Silver) y analíticos (Gold) con trazabilidad completa
2. **Optimización de formato:** Conversión exitosa de CSV a Parquet mejorando rendimiento de consultas gracias al nuevo formato
3. **Cálculo distribuido de KPIs:** Implementación eficiente de los indicadores técnicos pedidos, además de otros adicionales.
4. **Pipeline automatizado:** Orquestación completa Bronze - Silver - Gold con actualización automática del Data Catalog
5. **Configuración centralizada:** Sistema parametrizable mediante `constants.py` que facilita cambios de configuración sin modificar código de Jobs
6. **Reproducibilidad garantizada:** Script maestro `run_all.py` permite desplegar y ejecutar todo el sistema desde cero en minutos
7. **Escalabilidad demostrada:** Arquitectura preparada para procesar más criptomonedas sin cambios estructurales significativos